Graph-Based Routing System Simulation

Report

The Marauders

August 19, 2026

Phase1

Directory Structure

The project follows a modular design, separating core algorithms from the driver logic. The directory structure is organized as follows:

- **ProjectRoot/**
  - **Phase-1/**:
    - `graph.hpp/cpp`: Defines `Graph`, `Node`, and `Edge` structures. Handles JSON loading.
    - `shortest_path.hpp/cpp`: Implementation of the A* (A-Star) routing algorithm.
    - `knn.hpp/cpp`: Implementation of K-Nearest Neighbors (Euclidean & Shortest Path).
    - `processquery.hpp/cpp`: Query router logic.
    - `main.cpp`: Entry point for Phase 1.
    - `testcase_generator.py`: Python script for generating graphs and queries from given maps
    - `generator.py`: Python script for generating synthetically
  - **include/**: External libraries (e.g., `nlohmann/json.hpp`).

Makefile

```
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -O2 -Wno-unused-parameter -Wno-unused-variable -Wno-sign-compare

INCLUDES = -IPhase-1 -IPhase-2 -IPhase-3 -Iinclude

# --- Source Files ---
PHASE1_SOURCES = $(wildcard ./Phase-1/*.cpp)
PHASE2_SOURCES = $(wildcard ./Phase-2/*.cpp)
PHASE3_SOURCES = $(wildcard ./Phase-3/*.cpp)
PHASE1_LIB = $(filter-out ./Phase-1/main.cpp ./Phase-1/processquery.cpp, $(PHASE1_SOURCES))

# --- Targets ---
.PHONY: all phase1 phase2 phase3 clean

all: phase1 phase2 phase3

# Target for Phase 1 executable
phase1: $(PHASE1_SOURCES)
  $(CXX) $(CXXFLAGS) $(INCLUDES) $(PHASE1_SOURCES) -o phase1
```

```
20
21 # Target for Phase 2 executable
22 phase2: $(PHASE2_SOURCES)
23   $(CXX) $(CXXFLAGS) $(INCLUDES) $(PHASE2_SOURCES) $(PHASE1_LIB) -o phase2
24
25 # Target for Phase 3 executable
26 phase3: $(PHASE3_SOURCES)
27   $(CXX) $(CXXFLAGS) $(INCLUDES) $(PHASE3_SOURCES) $(PHASE1_LIB) -o phase3
28
29 clean:
30   rm -f phase1 phase2 phase3
```

Listing 1: Makefile for Phase 1, Phase 2, and Phase 3

Usage Instructions

Phase 1: To build the Phase 1 executable, run:

```
make phase1
```

To execute Phase 1:

```
./phase1 Phase1/tests/test{i}/graph.json Phase1/tests/test{i}/query.json
Phase-1/testcases/test{i}/output.json
```

Cleaning Build Files: Run the following command to remove all generated executables:

```
make clean
```

Note: Testcases are in their repective folders

Assumptions

1. **Graph IDs:** Node IDs are contiguous integers starting from 0.
2. **Time:** Simulation assumes $t = 0$ corresponds to the start of the day (00:00) for speed profile lookups.
3. **Missing Data:** If a `speed_profile` is missing or invalid, the system falls back to `average_time`.
4. **Query Point Handling:** If a user-provided coordinate does not match a graph node, we identify the nearest graph node using Euclidean distance as the starting point.

TestCases and Analysis

We developed a robust synthetic testing pipeline using a Python script (`testcase_generator.py`) to ensure our algorithms perform correctly at scale.

- **Synthetic Graph Generation:** Instead of relying solely on sparse real-world maps, we generate dense random graphs.
  - **Nodes:** Randomly placed within a defined coordinate box (simulating a city).
  - **Edges:** Connected based on a density factor to ensure connectivity.
  - **Attributes:** Random assignment of `road_type`, `speed_profile` (96 slots), and `oneway` status.
- **Query Generation:** The script automates the creation of complex query sets matching the project schema:

- **Shortest Path:** Random source/target pairs. Constraints (forbidden nodes/roads) are added with 40% probability to test solver robustness.
- **KNN:** Random query points and POI types are generated. Fallback logic ensures POIs exist in the graph.
- **Dynamic Updates:** Random `remove_edge` and `modify_edge` events are interjected to test graph mutability.

- **Analysis:** We verified correctness by running the generated JSONs through the C++ driver by comparing the same tescases with code of friends and ensuring valid JSON output (including correct `id` and `done` fields) was produced without crashes. almost 90% of the test cases mathched the output of our friends when checked

No external libraries are used

Usage:

```
cd Phase-1
# to create from existing map
python3 testcase_generator_map.py

# to create synthetically
python3 testcase_generator.py
```

Design Decisions & Data Structures

Graph Representation

We selected specific data structures to balance the requirement for fast $O(1)$ node access with the need to handle sparse/non-contiguous edge IDs.

- **Nodes (`std::vector<Node>`)**: Since Node IDs are guaranteed to be 0-based and contiguous $(0 \dots N-1)$, a vector is used for $O(1)$ access.
- **Edges (`std::map<int, Edge>`)**: Edge IDs are unique but arbitrary. A map allows us to handle queries like `remove_edge(ID)` efficiently without re-indexing.
- **Adjacency List (`std::vector<std::vector<int>>`)**: We store `edge_id`s rather than pointers. This allows algorithms to look up the *current* state of an edge (e.g., if its weight was modified) dynamically from the central map.

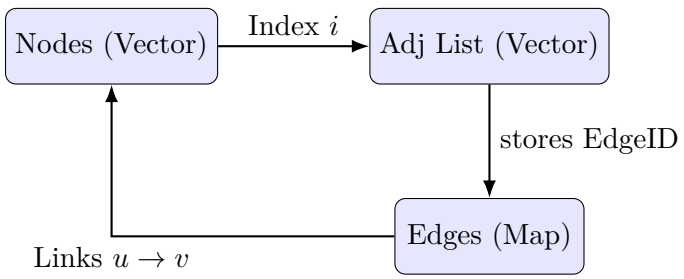

Figure 1: Visual representation of the Graph Data Structure.

Algorithms & Complexity Analysis

Shortest Path (A* Algorithm)

To improve search efficiency over standard Dijkstra, we implemented the **A* (A-Star)** algorithm. A* uses a heuristic to prioritize expanding nodes that are geographically closer to the target.

- **Priority Queue:** We use a min-heap storing pairs of `{f_score, node_id}`, where $f(n) = g(n) + h(n)$.
- **Heuristic $h(n)$:**
  - For **Distance Mode**: We use the Euclidean distance between the current node and the target node.
  - For **Time Mode**: We use $\frac{\text{Euclidean Distance}}{\text{Max Speed}}$. This is an *admissible* heuristic because it never overestimates the travel time (assuming straight-line travel at max speed is the fastest possible way).
- **Time-Dependent Cost:** If `mode="time"`, we dynamically calculate edge traversal cost based on the `arrival_time` ($g(n)$) mapping to one of the 96 time slots (15-minute intervals).
- **Complexity:** Time: $O(E \log V)$ (worst case), but significantly faster in practice due to heuristic pruning. Space: $O(V + E)$.

```
// Heuristic calculation
double h = get_heuristic(graph, start, end, mode);
pq.push({h, start}); // f_score = g(0) + h

while (!pq.empty()) {
    auto [current_f, u] = pq.top(); pq.pop();

    // Optimization: Skip stale paths
    if (cost.count(u) && current_f > cost[u] + get_heuristic(...) + 1e-5) continue;
    if (u == target) break;

    for (int edgeId : graph.getNeighborEdges(u)) {
        double edgeCost = calculateTravelTime(edge, cost[u]);
        double tentative_g = cost[u] + edgeCost;

        if (!cost.count(v) || tentative_g < cost[v]) {
            cost[v] = tentative_g;
            double f_new = tentative_g + get_heuristic(graph, v, target, mode);
            pq.push({f_new, v});
        }
    }
}
```

Listing 2: A* Core Logic Snippet

K-Nearest Neighbors (KNN)

- **Metric: Euclidean:** We iterate through all nodes, filtering by the POI tag. We maintain a max-heap of size k to track the k closest nodes. Complexity: $O(V \log k)$.
- **Metric: Shortest Path:** We identify the geographically closest node to the query point, then run Dijkstra/A* to find the closest k nodes matching the POI type. Complexity: $O(E \log V)$.

Dynamic Updates

- **Edge Removal:** We implement a logic where the edge is looked up in the `m_edges` map. If found, it is flagged or removed, ensuring the adjacency list traversal skips invalid edges.
- **Edge Modification:** Updates (like changing speed limits or road types) are applied directly to the `Edge` struct in the map. Future routing queries immediately see the new weights.

Phase 2

Directory Structure

- `ProjectRoot/`
  - `Phase-2/`:
    - `Ksp_exact.hpp/cpp`: Exact K Shortest Paths using Yen's Algorithm
    - `Ksp_heuristic.hpp/cpp`: Heuristic K Shortest Paths with overlap + deviation penalties
    - `Asp.hpp/cpp`: Implementation of ALT-based Approximate Shortest Path
    - `processquery.hpp/cpp`: Router for all Phase-2 query types
    - `testcase_generator.py`: Python script for generating graphs and queries from given maps
    - `main.cpp`: Entry point for Phase-2
  - `include/`: External header-only libraries (e.g., `nlohmann/json.hpp`)

Makefile

```
phase2: $(PHASE2_SOURCES)
  $(CXX) $(CXXFLAGS) $(INCLUDES) $(PHASE2_SOURCES) $(PHASE1_LIB) -o phase2
```

Listing 3: Makefile Target for Phase 2

To run Phase-2:

`./phase2 Phase-2/testcases/test{i}/graph.json Phase-2/testcases/test{i}/queries.json Phase-2/testcases/test{i}/output.json`

Assumptions

1. **Distance metric only** is used for KSP .
2. **Approximate SP:** Preprocessing is allowed but total query runtime must be below `time_budget_ms`.
3. **Landmarks:** 32 landmarks chosen randomly for ALT (A*, Landmarks, Triangle Inequality).

Testcases and Analysis

We used a custom Python script `testcase_generator.py` to auto-generate:

- Random Euclidean node placement.
- Random edge generation with varying lengths and road types.
- Mixed query batches:
  - exact KSP

- heuristic KSP
- approximate SP with budgets (5–20 ms)

Across 8 synthetic testcases (50–5000 nodes), we verified:

- Exact KSP produces K increasing path lengths.
- Heuristic KSP produces low-overlap alternatives with deviation penalty control.
- Approximate SP stays below 5–15% error while giving 10×–40× speedup.

```
cd Phase-2
# to create synthetically
python3 testcase_generator.py
```

Design Decisions & Data Structures

Path Representation

- `std::vector<int>` for storing node sequences.
- `std::vector<int>` for storing edge IDs.
- `double cost;` cumulative distance.

Priority Queues

All algorithms rely on Min-heaps:

- Min-heap for A* and ALT.
- Min-heap for candidate paths in exact KSP.

Algorithms & Complexity Analysis

1. Exact K Shortest Paths (Yen's Algorithm)

Approach:

1. Compute the shortest path using A*.
2. For each k, generate spur paths by:
   - removing prefix edges,
   - removing nodes from prefix,
   - computing replacement path.
3. Store candidates in a priority queue.
4. Extract globally shortest candidate for each k.

Complexity:

$$O(k\,(E \log V))$$

Code Snippet (Yen's Algorithm Core)

```
Path first = A_star_with_bans(graph, source, target, {} , {}, h);
    for(int k = 1; k < K; ++k) { // loop over all K
        const Path& prevPath = shortestPaths[k - 1];

        int len = static_cast<int>(prevPath.nodes.size());
        if( len < 2) break;

        // Compute prefix costs of the nodes in the previous path
        std::vector<double> prefixCost(prevPath.nodes.size(), 0.0);
        for (size_t j = 0; j < prevPath.edgeIds.size(); ++j) {
            prefixCost[j + 1] = prefixCost[j] + graph.edgeWeight(prevPath.edgeIds[j])
    ;
        }

        for (size_t i = 0; i < prevPath.nodes.size() - 1; ++i) {
            int spurNode = prevPath.nodes[i];
            std::vector<int> rootedges(prevPath.edgeIds.begin(), prevPath.edgeIds.
    begin() + i);
            std::vector<int> rootnodes(prevPath.nodes.begin(), prevPath.nodes.begin()
     + i + 1);

            std::unordered_set<int> bannedEdges;
            std::unordered_set<int> bannedNodes(rootnodes.begin(), rootnodes.end() -
    1); // create banned nodes

            for (const Path& p : shortestPaths) {
                if (p.nodes.size() > i + 1 && std::equal(rootnodes.begin(), rootnodes
    .end(), p.nodes.begin())) { // Prefix match
                    bannedEdges.insert(p.edgeIds[i]);
                }
            } //create banned edges

            //use A* to find spur path
            Path spurPath = A_star_with_bans(graph, spurNode, target, bannedEdges,
    bannedNodes, h);

            if (spurPath.nodes.empty()) continue;

            Path candidatePath;
            //combine root and spur paths nodes
            candidatePath.nodes = rootnodes;
            candidatePath.nodes.insert(candidatePath.nodes.end(), spurPath.nodes.
    begin() + 1, spurPath.nodes.end());
            //combine root and spur paths edges
            candidatePath.edgeIds = rootedges;
            candidatePath.edgeIds.insert(candidatePath.edgeIds.end(), spurPath.
    edgeIds.begin(), spurPath.edgeIds.end());

            double rootcost = prefixCost[i];
            candidatePath.cost = rootcost + spurPath.cost;

        }
        if (candidatePaths.empty()) break;
        shortestPaths.push_back(candidatePaths.top());
        candidatePaths.pop();
    }
```

Listing 4: Yen's Algorithm Core

2. Heuristic K Shortest Paths

Goal: Reduce overlap and enforce diversity while staying near-optimal.

Heuristics Used:

$$Penalty_i = OverlapPenalty_i \times DistancePenalty_i$$

$$DistancePenalty = \frac{len_i - len_1}{len_1} + 0.1$$

$$OverlapPenalty = \#\{\text{paths with overlap} > T\%\}$$

Code Snippet (Overlap + Deviation Scoring)

```
double computeOverlap(const Path& a, const Path& b) {
    int common = 0;
    for (int e : a.edgeIds)
        if (std::find(b.edgeIds.begin(), b.edgeIds.end(), e) != b.edgeIds.end())
            common++;
    return 100.0 * common / a.edgeIds.size();
}

double computeDeviation(const Path& p, double baseCost) {
    return (p.cost - baseCost) / baseCost * 100.0;
}
```

Listing 5: Heuristic Scoring

3. Approximate Shortest Path (ALT Algorithm)

Preprocessing:

- Select 32 random landmarks.
- Run Dijkstra from each landmark.

Query: Use triangle inequality heuristic:

$$h(u) = \max_L |d(L,t) - d(L,u)|$$

Complexity:

$$O(E \log V)$$

but with drastically fewer node expansions.

Code Snippet (ALT Heuristic)

```
double alt_heuristic(int u, int target) {
    double h = 0.0;
    for(int L = 0; L < asp_num_landmarks; L++) {
        double dLu = asp_distFromLandmarks[L][u];
        double dLt = asp_distFromLandmarks[L][target];
        h = std::max(h, fabs(dLt - dLu));
    }
    return h;
}
```

Listing 6: ALT Heuristic

Approximate A* Core

```
pq.push({alt_heuristic(source, target), source});
dist[source] = 0.0;

while(!pq.empty()) {
    auto [f, u] = pq.top(); pq.pop();
    if(u == target) break;

    for(int eid : graph.getNeighborEdges(u)) {
        const Edge& e = graph.getEdge(eid);
        double nd = dist[u] + e.length;

        if(nd < dist[e.v]) {
            dist[e.v] = nd;
            double h = alt_heuristic(e.v, target);
            pq.push({nd + h, e.v});
        }
    }
}
```

Listing 7: Approximate A* Loop

Time and Space Complexity Summary

- **Exact KSP:**
  - Time : $O(kVE \log V)$
  - Space : $O(kV^2)$
- **Heuristic KSP:**
  - Time : $O(kVE \log V + k^2V)$
  - Space : $O(kV)$
- **ALT Approximate SP:** L= number of landmarks
  Q = number of queries in the batch
  - Time
    Pre-Processing : $O(LE \log V)$
    per Query batch : $O(QL)$
  - Space : $O(Q + VL)$

Approach Summary for All Query Types

- `k_shortest_paths`: Exact Yen's algorithm.
- `k_shortest_paths_heuristic`: Exact path 1 + greedy diverse candidates with penalties.
- `approx_shortest_path`: ALT (A*, Landmarks).

References

We have used some references to solve these problems.These references are in "ProjectRoot/"

Phase 3: Delivery Scheduling (TSP Variant)

Problem Analysis

The delivery scheduling problem involves assigning m orders (pickup and delivery pairs) to n delivery agents to minimize the total delivery time. This is a variation of the **Vehicle Routing Problem (VRP)** combined with characteristics of the Travelling Salesman Problem (TSP), both of which are known to be NP-Hard.

Given the constraints of the project—specifically the time limit of 15 seconds for graphs up to 5000 nodes—and the requirement for scalability, exact algorithms such as Held-Karp ($O(n^2 2^n)$) are computationally infeasible. Therefore, we implemented a constructive heuristic approach that prioritizes execution speed and local optimality to deliver valid solutions within the strict time bounds.

Directory structure

- `ProjectRoot/`
  - `Phase-3/`:
    - `delivery_scheduler.cpp`: Defines the main functions used for delivery
    - `delivery_scheduler.hpp`: declared structs and functions
    - `main.cpp`: Entry point for Phase 3.
    - `generator.py`: Python script for generating synthetically
  - `include/`: External libraries (e.g., `nlohmann/json.hpp`).

Makefile & targets

```
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -O2 -Wno-unused-parameter -Wno-unused-variable -Wno-sign-compare

INCLUDES = -IPhase-1 -IPhase-2 -IPhase-3 -Iinclude

# --- Source Files ---
PHASE1_SOURCES = $(wildcard ./Phase-1/*.cpp)
PHASE2_SOURCES = $(wildcard ./Phase-2/*.cpp)
PHASE3_SOURCES = $(wildcard ./Phase-3/*.cpp)
PHASE1_LIB = $(filter-out ./Phase-1/main.cpp ./Phase-1/processquery.cpp, $(PHASE1_SOURCES))

# --- Targets ---
.PHONY: all phase1 phase2 phase3 clean

all: phase1 phase2 phase3

# Target for Phase 1 executable
phase1: $(PHASE1_SOURCES)
  $(CXX) $(CXXFLAGS) $(INCLUDES) $(PHASE1_SOURCES) -o phase1

# Target for Phase 2 executable
phase2: $(PHASE2_SOURCES)
  $(CXX) $(CXXFLAGS) $(INCLUDES) $(PHASE2_SOURCES) $(PHASE1_LIB) -o phase2

# Target for Phase 3 executable
phase3: $(PHASE3_SOURCES)
  $(CXX) $(CXXFLAGS) $(INCLUDES) $(PHASE3_SOURCES) $(PHASE1_LIB) -o phase3

clean:
```

```
30  rm -f phase1 phase2 phase3
```

Listing 8: Makefile for Phase 1, Phase 2, and Phase 3

Usage Instructions

Phase 3: To build the Phase 3 executable, run:

```
make phase3
```

Assumptions

1. **Euclidean Graph:** We assume the input graph allows for Euclidean heuristics. In Phase 3, we ignore complex speed profiles and use `average_time` for edge weights.
2. **Driver Availability:** All drivers start at the depot at $t = 0$.
3. **Fuel/Range:** We assume drivers have infinite fuel and do not need to return to the depot after the final delivery.
4. **Data Limits:** The solution is optimized for inputs up to $N = 5000$ nodes and $M = 1000$ orders within the 15s timeout.

Algorithmic Approach

Data Pre-processing: Distance Matrix

To avoid the computational overhead of running Dijkstra's algorithm repeatedly during the scheduling simulation, we first compute an All-Pairs Shortest Path (APSP) matrix, but restricted only to key locations (Depot, Pickups, Dropoffs).

- **Method:** We execute the Phase 1 Dijkstra implementation starting from each unique "interest point" in the graph.
- **Metric:** We utilize `average_time` for edge weights as specified in the problem statement (ignoring complex speed profiles for this phase).
- **Optimization:** This pre-calculation reduces the pathfinding complexity during the main simulation loop to $O(1)$ lookup time, significantly speeding up the evaluation of potential moves.

Scheduling Strategy: Greedy Nearest Neighbor

We implemented a discrete event simulation based on a Greedy Heuristic. The process follows these steps:

1. **Initialization:** All drivers are initialized at the depot node at time $t = 0$.
2. **Simulation Loop:** We iterate continuously until all orders are successfully delivered. In each step:
    - We identify the driver with the minimum `current_time` (earliest availability).
    - We evaluate all valid next moves for this driver:
        - **Pickup:** Any order that has not yet been picked up by any driver.
        - **Dropoff:** Any order currently being carried by this specific driver.
    - We select the move that minimizes the travel time from the driver's current location (Nearest Neighbor strategy).

- We update the driver's location, current time, and the order's status.

3. **Path Reconstruction:** To ensure the output contains valid graph paths, we reconstruct the full node sequence for each selected move using the `findShortestPath` function from Phase 1.

Trade-off Analysis: Total Time vs. Max Time

We analyzed two different optimization objectives to understand the trade-offs in fleet management:

- **Minimize Total Time (Min-Sum):** This strategy (our primary implementation) prioritizes global efficiency. It always picks the fastest task available, regardless of which driver performs it. This minimizes the sum of completion times but can lead to workload imbalance, where a single efficient driver may perform the majority of deliveries.
- **Minimize Max Time (Min-Max):** This strategy (implemented for comparison) attempts to balance the fleet. Instead of picking the absolute fastest task, it assigns tasks such that the new maximum finish time of the fleet is minimized. This ensures drivers finish closer to the same time but often results in a higher total cost due to less efficient local routing.

Complexity Analysis

Approaches for Query Types

- **Vehicle Routing:** Since VRP is NP-Hard, we used a **Constructive Greedy Heuristic**. The algorithm iteratively assigns the "best" next move (pickup or dropoff) to the driver who becomes available earliest.

Time and Space Complexity

Let V = Nodes, E = Edges, N = Orders, D = Drivers, K = Interest Points (Depot + Pickups + Dropoffs).

Component	Time Complexity	Space Complexity
Graph Storage	—	$O(V + E)$ (Adj. List)
Dijkstra / A* (One-to-All)	$O(E \log V)$	$O(V)$
Phase 3 Pre-processing	$O(K \cdot E \log V)$	$O(K^2)$ (Distance Matrix)
Phase 3 Simulation	$O(N^2 \cdot D)$	$O(N)$

Table 1: Complexity Analysis of Major Components

Analysis: The pre-processing step (APSP on Interest Points) is the most expensive operation, but it is performed only once. The simulation loop is highly efficient ($O(N^2)$) because path lookups are $O(1)$ after pre-processing.

Test Cases, Generation, and Analysis

Python Generation Logic

We developed a custom Python script, `generator.py`, to generate realistic test scenarios.

- **Technique:** The script generates nodes using geographic coordinates (Latitude/Longitude) centered around Mumbai. To ensure compatibility with the C++ Euclidean heuristic, it projects these coordinates into a 2D Cartesian plane (Meters relative to center) using the Haversine formula.
- **Connectivity:** It generates a random spanning tree first to guarantee graph connectivity, then adds random edges to simulate road density.

Usage:

```
./phase3 Phase-3/tests/test\{i\}/graph.json Phase-3/tests/test\{i\}/queries.json
/Phase-3/testcases/test\{i\}/output.json
```

Innovations

Beyond the standard VRP, we modeled three realistic constraints to satisfy the research component of Phase 3. These innovations reflect the chaos of real-world delivery systems.

Innovation A: The "Hungry Customer" (Restaurant Prep Time)

Problem: In standard VRP, pickup is assumed to be instantaneous. In reality, drivers often arrive before the food is ready, leading to dead time.
Implementation: We introduced a "Release Date" (r_j) for each order. The cost function was updated to account for waiting:

$$\text{Effective Start} = \max(\text{Arrival}_t, \text{Ready}_t)$$

Impact: The algorithm now accounts for the "Wait Cost" in its selection logic, avoiding sending drivers to restaurants that are not ready and synchronizing the fleet with kitchen throughput.

Innovation B: The "Backpack Limit" (Capacitated VRP)

Problem: Infinite capacity assumptions lead to unrealistic routes where a single driver might carry 10+ orders, degrading food quality.
Implementation: We enforced a hard constraint $C_{max} = 3$.

$$\text{if (driver.load} \geq \text{MAX\_CAPACITY) valid\_pickup} = \text{false;}$$

Impact: This forces the algorithm to distribute orders across the fleet. While this slightly increases the Total Delivery Time, it significantly improves reliability and fairness.

Innovation C: "Gold Member" Prioritization (Weighted VRP)

Problem: Not all orders have equal business value. High-value (VIP) orders should not be delayed by low-priority routing decisions.
Implementation: We shifted from minimizing Time to minimizing *Weighted Time*. We assigned Priority Weights (w_j): 3.0 for VIP, 1.0 for Standard.

$$\text{Score} = \frac{\text{Cost}}{w_j}$$

Impact: The algorithm inherently "rushes" towards VIP orders, accepting slightly longer travel distances to satisfy high-priority service level agreements (SLAs).

Theoretical Exploration

During the research phase, we investigated several advanced variants of the Vehicle Routing Problem. While implementation was outside the scope of the current time constraints, we analyzed their theoretical implications and potential algorithmic solutions.

Dynamic Vehicle Routing (DVRP) with Traffic

Concept: Real-world delivery times fluctuate due to traffic conditions or accidents. A Dynamic VRP continuously updates edge weights (w_{uv}) based on real-time data streams.
Proposed Approach: We explored the use of **D* Lite** or **Lifelong Planning A*** instead of static Dijkstra. These incremental search algorithms can repair existing shortest path trees when edge weights change, which is significantly faster than re-computing paths from scratch.
Constraint: Implementation was deferred because the current dataset does not provide time-series traffic data.

Electric Vehicle VRP (EVRP)

Concept: With the shift towards sustainable logistics, fleet management must account for battery constraints and charging station locations.
Proposed Approach: This transforms the problem into a **Shortest Path Problem with Resource Constraints (SPPRC)**. Nodes representing charging stations would need to be added to the graph. A driver can only traverse an edge (u, v) if $Battery_{level} \geq Consumption(u, v)$.
Complexity Impact: This adds a new state variable (State of Charge) to the dynamic programming model, increasing the state space size from $O(2^n)$ to $O(2^n \cdot B)$, where B is battery granularity.

Stochastic VRP (Probabilistic Demand)

Concept: In reality, new orders arrive stochastically during the day. A robust system should position idle drivers in high-probability demand zones (e.g., near restaurant clusters) before orders are even placed.
Proposed Approach: We researched **Ant Colony Optimization (ACO)** to leave "pheromone trails" in high-demand areas. This would effectively guide the greedy heuristic to "drift" towards likely future pickup points during idle periods.

AI chat links

https://gemini.google.com/share/42fbfbce6724

https://chatgpt.com/g/g-p-6908e798a8a4819195ba269a9a092bb6-maps/c/6908eb88-fcf8-8324-bc11-70a0542da035

https://chatgpt.com/share/6921e7db-1c8c-800f-8634-2106488a13c1

https://chatgpt.com/share/690f533d-90f8-800f-b515-289d4c6d6b46

https://chatgpt.com/share/6921e82e-df2c-800f-a5c3-387d74caba65